ArchIntel dynamic analysis: user flow

> **Phase 3 runtime update (4 October 2026):** For current session setup, instrumentation preview/exclusions, strict identity declarations, quality diagnostics and recovery, read Phase 3 user flow and developer handoff. Validation results include a new real PetClinic capture; older results below remain historical.

Purpose

Static analysis describes possible architecture. Dynamic analysis records what was observed while your application ran. ArchIntel combines these without replacing the static graph. This delivery is a local prototype with authenticated workspaces; run the application and ArchIntel on the same machine for the simplest setup.

```
Analyze compiled JAR/WAR statically
  -> Choose the matching completed static baseline
  -> Create a runtime session
  -> Configure the OpenTelemetry Java agent in the application process
  -> Start the session, then launch the application
  -> Exercise representative flows
  -> Observe real trace/span counts and runtime graphs
  -> Inspect mapping and coverage gaps
  -> Exercise additional flows
  -> Compare static, runtime and unified evidence
  -> Review readiness with observation confidence
  -> Stop the session and export its report
```

1. Start ArchIntel

In PowerShell:

```
cd 'C:\Users\afrin\OneDrive\Desktop\vfunction'
.\Start-ArchIntel.cmd
```

Keep the launcher window open. Open http://127.0.0.1:8000 in Chrome or Edge. A browser page alone does not start the server. If it reports connection refused, read the launcher output and `.data/api.err.log`.

For a fresh installation, run `scripts/setup.ps1` first. The locked Python dependencies now include OpenTelemetry protobuf support. A JDK and the jQAssistant exporter are needed for static analysis; the application also needs a suitable Java runtime.

2. Establish the static baseline

Use **Applications -> Analyze application** to upload your compiled application. Wait until analysis is completed. Use the same application build for static and runtime analysis: identical class names from a different version do not prove matching code.

New static analyses include method identities and explicit supported entry-point annotations. Older snapshots remain readable, but method and entry-point coverage may show **Unavailable**. Run a new static analysis when you need those denominators.

PetClinic is unusual: its application classes are under `org.springframework.samples.petclinic`, while the default noise filter excludes `org.springframework.`. The provided validation script explicitly includes that application namespace and excludes

`org.springframework.boot.` loader classes. It stores a real static analysis in the workspace without changing the default settings or previous snapshots. For another application with an overlapping framework namespace, review `EXTERNAL_PREFIXES` before scanning rather than accepting an empty or misleading graph.

3. Create a session

Open **Dynamic analysis** in the existing sidebar.

1. Choose the application and its completed static baseline.
2. Name the observation, for example `Owner flows - local`.
3. Choose an environment label such as LOCAL, TEST or STAGING.
4. Keep **BOUNDARY** as the default instrumentation depth. STANDARD adds statically important classes. DEEP explicitly selects broader application methods and is not the default.
5. Optionally enter comma-separated application package prefixes, for example `com.example.`.
6. Click **Create runtime session**.

Creating a session does not execute the uploaded application. ArchIntel displays a one-time bearer token inside the connection environment variables. Copy those settings into the terminal that will run your application, then hide them. They are not stored in browser local storage. **New token** revokes the old token and provides a replacement; update/restart the application's exporter accordingly.

4. Connect the Java agent

Use the official OpenTelemetry Java agent. The prepared workspace has version 2.31.1 at `.runtime/otel/opentelemetry-javaagent.jar`. `scripts/setup-runtime-demo.ps1` downloads that pinned version and builds the optional PetClinic sample.

Copy the generated environment-variable commands from the UI. They include:

- An application/session-specific OTLP traces endpoint.
- `http/protobuf` transport.
- A session bearer-token header.
- Disabled metric and log export.
- Explicit selected application class/method names, derived from the static inventory.

Then launch your application with the agent. For the prepared workspace JDK:

```
$javaDirectory = (Get-ChildItem '.runtime/java' -Directory | Select-Object -First 1).FullName
& "$javaDirectory/bin/java.exe" '-javaagent:.runtime/otel/opentelemetry-javaagent.jar' -jar 'C:/path/to/your-application.jar'
```

For a WAR deployed in a servlet container, configure the agent on that container's JVM according to its launch mechanism. Do not use `java -jar` on a non-executable WAR. Keep the application's port different from ArchIntel's port 8000. The validation sample uses 18080.

The generated method plan uses explicit method names, not `Class[*]`, because the verified agent's parser rejects that wildcard syntax. Constructors and synthetic method names are not included. Interfaces, generated proxies and uninstrumented paths may remain partially observed.

5. Start observation and run workload

Click **Start** for the session before launching the instrumented application. The session enters WAITING_FOR_AGENT. There is no fake agent handshake: **RECEIVING_DATA** appears only after accepted telemetry arrives. A quiet application may show DISCONNECTED after 60 seconds without new telemetry even while its JVM is healthy.

Run manual actions, integration tests or API requests that exercise important paths. Counts update automatically. A new session starts with zero traces; loading the dashboard does not create application activity. Repeated delivery of the same span is deduplicated within a session.

Sampling and selected instrumentation limit what can be observed. The generated sample configuration uses a parent-based always-on sampler; assess overhead and sampling before applying it outside local testing. No production overhead percentage has been established.

6. Inspect the evidence

Tab	What it provides
Coverage	Separate class, method, entry-point, executable-dependency and domain metrics; explicit unavailable denominators; domain observation gaps.
Graph	Domain, package and class views; runtime or unified evidence; click an edge for retained sanitized span links.
Mapping	EXACT, NORMALIZED, PROXY_RESOLVED or UNRESOLVED identities and the matching rule evidence.
Flows	Request groups by entry route and participating classes, with trace/error counts and entry-span duration. Class lists are not complete call sequences.
Database	Available database system/resource/operation and directly supported caller evidence. Unknown values stay unknown.
Modernizatio n	Preserved structural score, separate runtime score, combined score, confidence, formula and review risks.
Assistant	Deterministic explanations based on the selected session's evidence, with source links. No LLM service is required.

Select multiple sessions only when they share the same application and static baseline. Runtime and unified graphs combine those observations; static mode shows the unchanged baseline. The runtime assistant currently uses the first selected session, as labelled in its heading; select one session when asking about a specific observation.

Graph displays are capped at 150 nodes and 1,000 edges. The report export contains the full bounded result. A missing parent span is reported; ArchIntel does not invent a call across the gap.

7. Interpret coverage and fusion correctly

- **STATIC_ONLY:** detected statically, not observed in the selected sessions. It does not mean unused.
- **DYNAMIC_ONLY:** observed at runtime without a matching static class-pair dependency. This may include proxies, reflection or external/resource observations.
- **STATIC_AND_DYNAMIC:** a class-pair dependency has both kinds of evidence. Inspect static types and spans for details; this is not proof that every static dependency type was exercised.

Class coverage uses distinct mapped application classes. Dependency coverage uses distinct directed static METHOD_CALL class pairs. Method coverage counts unambiguous owner/method observations against known static signatures; overloaded names without signature evidence remain unobserved. Entry-point coverage uses explicit supported method annotations, not every possible inherited or composed route.

Domain coverage means at least one relevant class in a domain was observed. Domain rows additionally distinguish UNOBSERVED, PARTIALLY_OBSERVED and OBSERVED class membership. Even all classes observed does not imply every branch or business flow was exercised. These are architectural observation metrics, not JaCoCo instruction/branch coverage.

Exercise an additional suggested flow and check the relevant metrics again. Recommendations are labelled SUGGESTED_SCENARIO and grounded in unobserved static classes; business steps are not inferred with certainty.

8. Review modernization

Structural scores remain unchanged. Runtime scores use configurable cohesion, cycle and persistence components. No mapped runtime calls means unknown runtime readiness, so the combined result retains the structural score. Low coverage reduces the displayed observation confidence; it does not subtract points from structural readiness.

The existing dependency-first modernization order is retained, with runtime evidence available for review. This is an explainable prototype heuristic, not calibrated extraction effort or an automated migration approval.

9. Stop and export

Stop the workload, allow the agent's batch exporter to drain, then click **Stop** for the session. ArchIntel produces a completed report and rejects subsequent ingestion. A completed session cannot be restarted; create another observation session.

Click **Export evidence** to download JSON. Completed reports remain available after sanitized span retention expires, but individual span links then return not found. Combining an expired session with another session is rejected because the retained report does not contain enough span-level evidence to recompute a trustworthy union.

By default, sessions are automatically closed after 24 hours and sanitized events from completed sessions are retained for a further 24 hours. A background sweep runs every 60 seconds while ArchIntel is running. Stop your Java application separately. Use `Stop-ArchIntel.cmd` to stop ArchIntel while preserving its database.

10. Reproduce the prepared real demo

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts/setup-runtime-demo.ps1
# Start ArchIntel in another terminal, then:
.runtime/python/tools/python.exe scripts/validate_runtime_petclinic.py
```

The script builds/uses the pinned public PetClinic sample, runs real static scanning, creates a session, starts the real Java agent, exercises owner then vet/pet/visit flows, verifies increased coverage and unchanged static evidence, stops the session and terminates the sample JVM. It does not execute uploaded customer artifacts. Results are recorded in `docs/runtime-validation.json` when all assertions pass.

Troubleshooting

Symptom	Check
Connection refused at port 8000	Start ArchIntel and keep its launcher open; inspect API logs.
WAITING_FOR_AGENT	Session started; agent attached to the actual JVM; generated environment set in that JVM's launch terminal; workload executed.
HTTP 401 from exporter	Session token/header is missing, rotated or belongs to another session.
HTTP 409	Session is completed/not started, or selected sessions have incompatible baselines.
Invalid trace method configuration	Use the newly generated explicit method plan and verified agent version.
Few mapped classes	Check package filtering, old static inventory, agent method selection and workload completeness. Framework spans alone may not expose application methods.

Symptom	Check
Dropped spans	Session reached its configured span cap; create a shorter/scoped observation and review sampling.
Unknown database resource	The agent did not provide a reliable collection name; ArchIntel does not parse/store SQL text to fabricate one.

Runtime ingestion requires its own session token. Management APIs now require an authenticated workspace identity and explicit permissions. See PHASE-1-USER-FLOW.md. Production SaaS deployment is not validated.

Optional local AI evidence selection

The runtime assistant can now use the Phase 2 AI gateway to select supported facts from the selected session. It retains evidence links and deterministic fallback. Enable a registered local provider and grant MODEL_USE_LOCAL alongside RUNTIME_READ; see PHASE-2-USER-FLOW.md. No generative model changes coverage or modernization scores.